

标书制作 Skill 全周期开发经验总结（专业版）

依据《经验总结写作提示词》v1.1 撰写。覆盖范围：**更长周期（2026-09 至今，全周期开发复盘）**，粒度按里程碑。说明：git 与 development-log 可追溯的工程起点为 **2026-10-01**（用户 9 月的早期讨论/需求未留下代码与日志记录），本文按“项目全周期（2026-10-01 ~ 2026-10-07）”取证，时间线以此为准。取证来源：dev/notes/development-log.md（1217 行全文）、git log（约 130 条提交）、三塔层实跑（2026-10-07 基线）。

1. 结论与量化结果

一句话结论：7 天连续开发，把“标书制作 skill”从 workbuddy 私有结构下的三个占位目录，做成一个**平台无关、可分发、数据隔离、测试金字塔 + CI/CD 闭环**的全流程标书制作系统（M1 素材库 / M2 模板库 / M4 招标解析 / M5 项目模板与商务标 / M7 交标检查 / 产物反馈 / M9 经验总结），期间落地了双通道解析、内容校验铁律、公司级回收站等 10 余项用户裁决的架构决策，并完成一次公网安全治理。

维度	计划/起点	实际/终点	备注
版本演进	v0.1（占位骨架）	v0.8.9 + M2/M4 v2.5/M5/M7/M9	10 次主要版本、13 个功能模块
里程碑	1 个（初始提交）	7 天 33+ 个 PR 里程碑	git log 约 130 条提交
测试用例数	0（无测试）	unit 242 / integration 71(13 skip) / e2e 26(1 skip)	全绿，实跑确认
CI	无	双平台 (ubuntu+windows) × Python 3.10–3.12 矩阵	4 个必需 check + CodeQL

维度	计划/起点	实际/终点	备注
交付模块	素材库	M1 素材库 / M2 模板库 / M4 招标解析 / M5 项目模板+商务标 / M7 检查 / M8 产物反馈 / M9 经验总结	全链路可用
关键耗时	—	日志可考约 20+ 小时	含 3.5h Word 损坏诊断、3h OCR 重写
生产部署	无	标书匠生产目录 独立 git + 自动部署 + 定时兜底	生产与开发隔离

2. 关键事件时间线

日期	事件	影响/决策
10-01	项目级化改造 v0.2 → v0.3	skill 改为项目级（仓库根即 skill），平台无关可分发；删除占位目录与 .workbuddy
10-01	架构定位澄清 v0.3.0	素材库+模板库=企业级、知识库=共享级；三层软件目录 v0.5.0 固化
10-01	测试金字塔 + CI/CD v0.7.0	L1/L2/L3 三塔层 55 例全绿；素材库不入仓；双平台 CI + tag 发布
10-02	M4 招标文件解析交付 v0.8.0	agent 解析双文本产物（投标要点+素材清单）；tender-* 6 命令；BOM 容错
10-02	M4 首次实战（示例河流 PDF）	90 页/76531 字符提取；项目名自动确定；原件幂等归档
10-02	M1 同名七连修 v0.8.2 → v0.8.7	同名判定语义从“关键字交集”演进到“按基名/完整职称名”
10-03	M2 模板库（首批 10 模板）	三段式占位规则 + 媒体剥离 v3；台账 +CLI 全套；152 测试全绿
10-03	M5 项目模板生成器	按“格式契约+素材清单+基础模板”动

日期	事件	影响/决策
	+ 反馈机制	态生成; fb-* CLI; 182 测试
10-03	M4 第二次实战 (示例化工园 docx)	保证金不免缴、业绩门槛 6000 万等关键差异识别; 7 件产物
10-04	M5 填充引擎 v1.0 → v1.1	简历 OCR + 组合图片插入 + 缺图清单; 正式商务标缺图归零
10-04	Word 损坏根因定位	sectPr 前置 + 悬空 footerReference 四轮诊断; _sanitize_copy 统一清理
10-04	12 条改进建议全部实施	docx_util 工具化/白名单/参数化/M7/word_open/台账单源/CI Win/冻结/拆分/PDF 解析/混合检索/docxtpl 评估
10-05	企业模板库定位四次澄清	模板库=规则库 (非直接可用模板); 项目模板=解析+素材清单+模板库动态生成
10-05	敏感信息治理	5 文档 gitignore; git-filter-repo 重写 105 commits; 公网零敏感词
10-05	CI/CD 体系 8 项迭代	手动触发/CodeQL/Dependabot/权限最小化/pip 缓存/日志 artifacts/失败开 Issue/fork 限定
10-06	WorkBuddy 诊断修复 + 版本真源统一	description 压缩、SKILL.md 删 version 字段 (git tag 为唯一真源)
10-06	公司级统一回收站	init-enterprise 建 <企业>/回收站; rm-project 移入; 30 天清理; 旧库自动迁移
10-07	M4 解析框架 v5 + 铁律 D25	9 模块框架; 合同不解析; 响应文件格式独立文件; 差异人工决策铁律
10-07	M4 v2.1 双通道机制 + 功能落地	通道 A 规则引擎 (tender-rule/fmt/diff) + 通道 B agent 语义; 差异逐项裁决
10-07	M4 v2.2 → v2.5	通道 B 提示词固化 v1.0; HTML 展示; 裁决纪律 v2.4; 8 条建议落实 v2.5
10-07	M9 经验总结写作提	三层提炼 + 带证据 5 Whys + 评分制自检

日期	事件	影响/决策
	示词固化 v1.1	+ Golden 回归；PDF 双版本产出

3. 开发过程总结

3.1 方法论：怎么做的（非只做了什么）

本项目建立并反复验证了一套可复用的开发方法论，**每一条都先被用户裁决、再被实践检验**：

1. **需求复述对齐铁律（先对齐→再建议→后执行）**：用户提出的任何需求，agent 先复述对齐理解、用户确认后再联网查建议、最后执行。实践效果：M2 模板库 4 问对齐、M5 图片尺寸 17 类逐条确认、M4 解析框架 9 模块多轮澄清——**把“做错方向的返工”前置为“做之前的澄清”**。
2. **七步开发套路**：需求对齐 → 契约文档 → 落代码 → 测试 → 操作手册 → 回填 module-contracts/SKILL.md → 推送。每轮开发都以“契约先行”开篇（M2-需求对齐 D1-D13、M4-需求对齐 v1.1→v2.5、产物反馈机制 D14-D16）。
3. **测试金字塔 + CI/CD 卡点**：unit/integration/e2e 三塔层从 55 例增长到 339 例；CI 双平台矩阵 + 分支保护（必须 PR + 4 个必需 check）+ 自动部署 + 自动打 tag + 失败自动开 Issue。**质量网不是事后补的，是从 v0.7.0 就搭的。**
4. **scripts/ 分包结构化**：薄壳入口 bidcraft.py + _shared/ 共享层 + 每模块子包 m<n>_<name>/，单文件红线 ~500 行。generator/filler/core 超限后拆分（⑦ 项），facade 保持对外 API 零变化。
5. **数据隔离**：素材库/模板库/项目数据全部在企业级/项目级目录（--root 指向），**素材库天然不入 git**；测试用 tempfile 临时根，绝不触碰真实素材库。
6. **对话检查点**：每轮对话在关键决策点设检查点（素材清单生成前先问用户、apply 前确认同名处理、双通道差异逐项裁决）。

3.2 关键架构 / 决策演进（为什么这样定）

- **项目级化改造 (10-01)**：为什么——workbuddy 私有结构不可分发，而用户要求“平台无关、可打包分发给其它智能体（workbuddy/hermes/trae work 等）”。

决策：仓库根即 skill，数据目录由运行时 --root 指向，脚本按需创建。**教训**：被 git 跟踪的目录删除必须 git rm 同步索引，否则留脏状态。

- **三层软件目录 (10-01)**：<软件根>/{企业}/企业级/素材库+模板库 + 项目级 + <软件根>/共享资源/知识库。为什么——企业间完全隔离、项目间完全隔离，知识库共享级支撑技术标。这是所有 M1/M2/M4/M5 数据落位的基座，后续 10 余个模块无一违反。
- **双通道解析机制 (10-07)**：通道 A 文档解析（规则引擎，确定性字段快准可调试）+ 通道 B agent 语义解析（9 模块开放性语义）。为什么——单靠规则引擎会漏评分细则等开放性内容，单靠 agent 会编页码/漏细节，两者互补；**实质差异必须列示并逐项交用户裁决 (v2.4)**，agent 只给建议不下结论。行业依据：AI 标书工具主流 = 规则引擎 + LLM 双驱动（SegmentFault 招投标结构化抽取/云境标书 AI 合规引擎/发改法规〔2026〕195 号）。
- **解析确认闸门 (v2.3)**：解析结果以单文件 HTML 展示（KPI 速览 + 9 模块明细 + 双通道差异表 + 红线红色块），用户核对确认后才进入素材清单环节。为什么——投标是高风险动作，解析完整性必须人工闸门，不能直接进下一步。
- **内容校验铁律 D25 (10-07)**：响应文件格式（招标原文）一字不改，是**内容地基**；企业模板库是**格式与排版契约**；项目模板引用企业模板后做内容校验，固定文字差异以招标原文为准，**先列差异清单人工确认后才更新**。为什么——招标原文是投标合法性的生命线，任何自动改写都是废标风险。
- **企业模板库定位四次澄清 (10-05)**：模板库 ≠ 可直接使用的模板，而是**规则库**（告诉项目模板如何设置占位/大小/位置）；项目模板 = 招标解析 + 已确认素材清单 + 企业模板动态生成；**已确认素材清单既是项目模板正确性的保证，也是商务标正确性的保证**。
- **公司级统一回收站 (10-06)**：所有删除动作（素材/项目）一律进 <企业>/回收站，默认保留 30 天，其他地方不再设回收站。为什么——删除是高风险动作，统一回收站给“误删恢复”留了窗口；旧素材库回收站自动迁移。

3.3 当前开发流程中的问题（流程级，非单点 bug）

对照《模块开发规范》七步套路，从日志提炼出以下**反复出现的流程摩擦**：

1. **需求澄清轮次过多，契约前置不够**：企业模板库定位经历 4 次修正（10-05 四轮），模板需求规格为多次口述（10-04 用户口述 12 条规则）。原因：需求涉及“模板库的定位/动态生成机制/内容校验边界”这类高阶语义，早期对齐只落到 D1-D13，未覆盖“模板库=规则库”这一本质定位。**改进：需求对齐文档应包含“本质定位”一节（这模块到底是什么、不是什么），并在每轮开始时复核。**
2. **对话检查点曾被跳过（v2.4 事件）**：HTML 解析文件含 5 项用户裁决项，但 agent 未在对话上下文逐项明示请求裁决，自行判定“实质一致/不列示/不并入”。原因：裁决纪律只存在于操作手册文字里，**没有硬性的“对话中必须逐项列出”检查**。**改进：v2.4 已固化“双通道差异必须逐项在对话上下文明示 + 每项附判定建议 + 裁决结果记入已裁决事项块”；更进一步应把“是否有未裁决项”做成落盘前的程序化校验。**
3. **静默 bug 类回归（不落盘/不同步）**：⑥ 暴露 fb.register 改内存索引不落盘（此前所有版本 bump 未真正写入）；v0.8.12 暴露 trash 只移文件不清台账；10-05 暴露 query 遇 keywords 脏数据（list）TypeError。共同原因：**“写后不回读”**——save 后没有立即回读断言。**改进：所有“写入权威源”的路径补“回读校验”单测；register/trash/query 已补。**
4. **真实数据依赖测试的跳过条件缺陷**：8 个 integration/e2e 用例在数据缺失时报错而非跳过（skip 条件只查企业根目录，未查格式契约/素材清单/项目模板前置）。原因：skip 守卫设计时按“环境存在性”而非“数据前置存在性”。**改进：已细化 _REAL_OK 前置检查；新写真实数据用例时，前置条件必须列出全部依赖数据文件。**
5. **单文件行数超限后才拆分**：generator 1180 行/filler 1113 行/core 1175 行，违反 500 行红线后由 ⑦ 项一次性拆包。原因：功能迭代期“先跑通”优先于结构约定。**改进：模块初版即按子模块规划（gen_text/gen_images/gen_tables/...），把拆分前置。**
6. **Word COM 环境类问题反复**：gen_py 缓存过期导致 access violation、Quit 不退出进程需 taskkill、explorer 句柄占用阻止目录重命名、Word 受保护视图阻塞自动化。共同原因：Windows 环境治理知识是“点状”的，每次踩坑后修复但未沉淀为环境检查工具。**改进：com_util.py 已统一 Word COM 封装（单实例+PID taskkill+等待句柄）；gen_py 清理可做成工具化命令。**

3.4 可复现的改进建议

1. **需求对齐模板增加“本质定位”节**：每个模块的需求对齐文档先写“这个模块是什么/不是什么、谁消费它的产物”，避免高阶语义澄清轮次（如模板库=规则库）。复现方式：作为《模块开发规范》附录模板。
 2. **裁决类检查点程序化**：把“双通道差异逐项裁决”从操作手册文字提升为**落盘前校验**（tender-parse 时若 diffs 非空且无 verdicts 记录则拒绝落盘）。复现方式：m4 校验逻辑扩展。
 3. **写后回读断言标准化**：所有 save/register 路径的测试强制加“读回断言”（save 后 load 对比）。复现方式：测试模板/开发规范加一条硬性约定。
 4. **真实数据用例前置清单化**：每个真实数据依赖用例的 skip 条件 = 依赖数据文件存在性清单（可配置）。复现方式：测试帮助函数 require_data(files)。
 5. **Word 环境检查工具化**：bidcraft env-check 子命令——检测 python-docx/Pillow/PyMuPDF/rapidocr/Word COM/gen_py 缓存/explorer 占用，一键诊断。复现方式：新模块（m_env_check），复用 com_util。
 6. **模板需求规格口述→文档化前置**：用户口述的模板规则（10-04 十二条）在制作前先整理成《模板需求规格.md》并由用户确认再动手。复现方式：M2 制作流程增加“规格确认”检查点。
-

4. 做对了什么（Continue）

4.1 需求复述对齐铁律

- **成绩**：7 天 33+ 里程碑无一次“方向性返工”；M4 解析框架 9 模块、M5 图片口径 17 类均由用户逐条确认后落地。
- **做法**：每轮开发第一步是复述对齐（M2 四问、M5 图片尺寸逐条、双通道方案评估 4 冲突+5 增强），用户确认后才联网查建议、执行。
- **经验（三层提炼）**：
 - 原来难在哪：招标解析/模板生成这类“需求半明确”任务，直接动手极易做错方向（如把模板库做成可直接用模板）。
 - 改了哪个关键环节：把“对齐”从一句口头确认升级为**文档化契约 + 对话检查点**（M4-需求对齐 v1.1 → v2.5、M2 D1-D16），每条决策有编号可回查。

- 换环境怎么照做：新项目（新模块/新需求）开工前必走“复述→确认→建议→执行”，且把确认结果写进需求对齐文档版本表。

4.2 测试金字塔 + CI/CD 闭环

- **成绩**：unit 242 / integration 71 / e2e 26（339 项全绿）；CI 双平台 × 3 Python 矩阵；main 分支保护（PR + 4 check + strict）；自动部署到生产 + 自动打 tag + 失败自动开 Issue 自动关闭（实测 #22 闭环）。
- **做法**：v0.7.0 从第一天搭三塔层；12 条建议中的 CI 硬化（Windows runner、pip 缓存、日志 artifacts、fork 限定）；10-05 部署自动化（self-hosted runner → 生产 git 拉取 + 冒烟 + 版本记录）。
- **经验**：测试金字塔的价值不在于数量，而在于**每层守一个风险**——unit 守纯逻辑（命名引擎 45+ 用例），integration 守存储层与模块间契约，e2e 守 CLI 全流程与退出码；CI/CD 的价值在于把“人记得检查”变成“机器强制检查”（分支保护让“直接 push main”物理不可行，实测 remote rejected ✓）。换环境：新仓库第一天就建三塔层 + 分支保护，后续每功能分支 PR 合并，成本远低于事后补质量网。

4.3 双通道解析 + 差异裁决机制

- **成绩**：通道 A 规则引擎 11 条确定性规则（tender-rule）+ 通道 B agent 语义 9 模块（提示词 v1.1 强制）双轨落地；tender-diff 自动比对；v2.4 裁决纪律固化；示例市 2026 实跑产出 5 项差异逐项裁决。
- **做法**：规则引擎负责确定性字段（编号/截止/保证金/联合体/实名制），agent 负责开放性语义（评分细则/废标红线/监理工作内容）；两者产出自动 diff（数字归一化消除单位后缀误报），差异列示 + 用户逐项裁决，裁决记入已裁决事项块。
- **经验**（三层提炼）：
 - 原来难在哪：单靠规则引擎漏开放语义（评分细则无法穷举），单靠 agent 会编页码/漏确定性字段（幻觉风险）。
 - 改了哪个关键环节：**让两个通道互补，并把“谁说了算”交给用户**——引擎给确定性、agent 给完整性、差异给裁决权；提示词 v1.1 分段精读流程进一步降低 agent 幻觉（严禁整段通读、逐章节交叉查重）。
 - 换环境怎么照做：任何“规则 + 语义”混合解析任务，都可以套“引擎抽确定性字段 + LLM 抽开放语义 + 自动 diff + 差异交人裁决”四步。

4.4 scripts 分包 + 数据隔离

- **成绩**：7 个模块子包 + _shared 共享层；单文件红线 ~500 行（拆包后最大 inbox.py 476 行）；素材库/模板库/项目数据全部在企业级目录，仓库零敏感数据（公网公开后 git-filter-repo 重写 105 commits，全历史敏感词 0 命中）。
- **做法**：薄壳入口调度 + facade 导出全部历史符号（对外 API 零变化）；--root/BIDCRAFT_LIB_ROOT 运行时指向；测试 tempfile 临时根。
- **经验**：分包 + 数据隔离是“平台无关可分发给其它智能体”的前提——**依赖运行时路径而非仓库占位**，使 skill 整体打包后在任何 agent 环境可用。换环境：新 skill 一律按“薄壳 + 子包 + 数据目录外置”起手。

4.5 提示词资产化（M9/M4 同一治理思路）

- **成绩**：M4 通道 B 提示词 v1.1（9 模块+铁律+JSON 骨架）与 M9 经验总结提示词 v1.1（三层提炼+评分制+Golden 回归）均版本化、有变更记录、可回滚；Golden 测试集 tests/fixtures/m4_golden 4 份样例 + test_m4_golden.py 结构回归。
 - **做法**：提示词 = 资产（不是一次性 prompt），进 references/、版本表、Golden 回归（改提示词前先跑 Golden 重写对比）。
 - **经验**：提示词治理与代码治理同构——**版本化 + 回归 + 可回滚**，否则提示词漂移无法审计。换环境：任何高价值提示词（解析/写作/审查）都按“固化→版本→Golden→回归”管理。
-

5. 踩坑与根因（Discontinue）

5.1 Word「文件可能已损坏」：悬空 footerReference（10-04，3.5h）

- **现象**：M5 图片占位可视化后，四分之一分段生成的 docx 全部被 Word 判损坏（com_error -2147352567）。
- **5 Whys（带证据）**：
 1. 为什么 Word 判损坏？→ document.xml 里存在指向不存在 part 的引用（Word 打开时校验失败）。

2. 为什么引用悬空？→ 招标原文章段落落在 w:pPr/w:sectPr 内带
`<w:footerReference r:id="rId12..rId19">`，深拷贝进新文档后 rId 在新
rels 中不存在。
 3. 为什么 footerReference 会被带进来？→ `_extract_blocks` 按段落块整体
deepcopy，未清理引用类元素。
 4. 为什么此前单块测试通过、分段失败？→ 悬空引用分布在 8 个块
(878/886/892/900/906/913/926/932) 横跨整个资格证明区间，单块测试恰好
不包含；四分之一分段必然包含 → **测试样本未覆盖真实分布**。
 5. 为什么没有更早发现？→ 旧模板由人工 Word/WPS 修订保存时被自动修复，
掩盖了问题；且此前无“交付前 Word 打开冒烟”检查（假信号：python-
docx 往返保存字节相同但 Word 拒绝，曾误判为 COM 进程假阳性）。
- **当时解法**：`_sanitize_copy` 深拷贝后统一清理——移除
headerReference/footerReference、移除示例图 w:drawing/w:object/w:pict、剥离其余
r:embed/r:id/r:link（超链接退化纯文本）；新增 `test_no_dangling_rel_refs`。
 - **下次如何避免**：M7 检查表第 4 行 `word_open` 常态化（每文件全新 DispatchEx 只读
打开）；任何 docx 生成/编辑路径交付前过 M7 四道。
 - **系统性模式**：Word OOXML 的“引用完整性”（rels 可解析性）是跨模块的交付
红线。

5.2 M1 同名判定语义七轮演进 (10-02, v0.8.2→v0.8.7)

- **现象**：同名（关键字相同 → 删除更新素材库）功能在真实素材上连续 7 轮修复：
①空关键字固定词子类不判 → ② `trash_old` 后旧台账行不清理 → ③职称证书等级
词误判 30 条 → ④注册证书类型词误判 3 条 → ⑤不同等级职称误判同名 → ⑥同
人不同评审年份证书误判 → ⑦退休证登记表误判。
- **5 Whys (多因分支)**：
 - 分支 1（判定过宽）：为什么职称证书误判 30 条？→ 关键字交集含等级
词“工程师”，与全部职称证书交集。→ 为什么等级词参与判定？→ 判定
函数未区分“类型词/等级词”与“实质标识”。→ 修复：排除
`PERSON_FIXED_WORDS` 后按专业交集。
 - 分支 2（标识粒度过粗）：为什么不同等级/不同年份证书仍误判？→ 判定
基准是“等级+专业”基名，未纳入 `final_name` 年份。→ 为什么基名计算忽

略 final_name? → 实现时认为 build_name 即最终名。→ 修复：优先以 final_name (_strip_page_suffix 后) 为基名。

- 分支 3 (子类语义混淆)：为什么退休证登记表被误判? → 退休证子类走了“同人同子类即同名”，与同批刚归档的退休证长期撞车。→ 为什么退休证是例外? → 退休证/职称证书有固定基名，简历/身份证等无独立标识。→ 修复：退休证与职称并列按基名比较。
- **当时解法**：每轮按真实批次暴露的边界加 L2 用例 (最终 30 例 integration)，_name_conflict_hit 统一判定 + _strip_page_suffix。
- **下次如何避免**：同名/去重类判定，先用真实台账全量回放再上线 (把“第 7 轮才遇到”前置为“第一轮就扫描全部存量”)。
- **系统性模式**：语义判定 (同名/分类/去重) 在真实数据上的边界远多于单测想象，必须“存量数据回放 + 用户裁决制”双保险。

5.3 荣誉证书日期 OCR 误读 (10-01, 用户 2 次修正)

- **现象**：第十三批 2 张“优秀监理企业”证书命名日期错误——20220501 应为 20230201 (误读年度)、20240501 应为 20240301 (误读月份)，用户逐字修正。
- **5 Whys (带证据)**：为什么日期错? → agent 未逐字核对证书落款 (荣誉年度 + 颁发年/月)，凭“看一眼”印象填。为什么没核对? → 归档流程没有“OCR 结果 vs 落款逐字比对”的强制检查点。为什么没有检查点? → 早期归档把 OCR 当可信输入，未意识到证书日期是“低置信字段”。
- **当时解法**：用户修正 + 台账同步 + 固化铁律“归档后必须与证书落款逐字核对”。
- **下次如何避免**：日期类字段 (荣誉年度/颁发日期) 一律按“低置信”处理，强制二次核对；简历 OCR 同样重写 3 轮 (rapidocr 优于 tesseract、y 聚类成行、身份证号第 17 位推断性别)。
- **系统性模式**：OCR/识别类字段是系统性易错点 (荣誉日期、姓名、证书编号、项目名拆分)。

5.4 Windows 编码 / BOM 系列 (贯穿全周期)

- **现象**：① PowerShell Set-Content -Encoding UTF8 写 JSON 带 BOM → json.loads 直接失败 (M4 v0.8.0 冒烟暴露)；② PowerShell 中文 stdout/stderr 常被吞 → 用文件回读；③ cmd 跑 UTF-8 bat 中文乱码 (GBK 才正常)；④ ps1 必须

UTF-8 BOM (Write 无 BOM 被 5.1 按 ANSI 解析) ; ⑤ 2>&1 重定向原生命令触发 NativeCommandError 假报错 (实际 exit 0) 。

- **5 Whys (多因分支)** : 为什么 BOM 会进来? → PowerShell 5.1 的 -Encoding UTF8 默认写 BOM, 而 Python open(utf-8) 不认 BOM。为什么 stdout 被吞? → Windows 控制台代码页与管道编码不一致。为什么反复踩? → **编码知识是点状的, 每次修一个具体症状。**
- **当时解法**: load_doc() 用 utf-8-sig 优先解码剥 BOM (md/json 一律 BOM 容错) ; 中文输出写文件再 Read; .cmd 用 GBK + chcp 936; ps1 用 BOM; 不用 2>&1 直接看 stderr + 查 exit code。
- **下次如何避免**: Windows 下所有文件读写统一约定——**Python 侧 utf-8-sig 容错, 脚本文件按目标解释器编码生成 (.ps1=BOM、.cmd=GBK) , 中文输出不依赖管道。**可工具化为 env-check 的编码检查项。
- **系统性模式**: Windows 编码/BOM 是跨模块第一大系统性坑。

5.5 静默 bug: 写后不回读 (10-04/10-05/10-06)

- **现象**: ⑥ 修复 fb.register 不落盘 (改内存索引、save 写旧数据——此前所有版本 bump 均未真正写入) ; v0.8.12 trash 只移文件不清台账; 10-05 query 遇 keywords list 脏数据 TypeError。
- **5 Whys (带证据)** : 为什么 register 不落盘? → register 里 idx = baseline_index(ent) 重新读盘, idx[rel] 与 rows 是两份独立解析的 dict, 改 idx[rel] 不影响 rows。为什么没有测试抓到? → 测试断言了返回值而未回读文件。为什么 save 不报错? → save 写的是未修改的旧 rows, 语法合法。
- **当时解法**: register 内索引基于同一份 rows 构建 + test_feedback_register 2 用例; trash 移文件后同步台账 (L2 断言) ; query 兼容 list keywords。
- **下次如何避免**: **所有“写入权威源”路径补回读断言** (save 后 load 对比) ; 新增“数据一致性巡检” (台账/磁盘/回收站三方对账) 。
- **系统性模式**: 静默不落盘/不同步是高风险模式——不报错但数据错, 比显式失败更难发现。

5.6 CI 云端红：缺运行依赖（10-05，历史 82 条记录多数 failed）

- **现象**：引入 docx_util 起 CI 一直红（ModuleNotFoundError: python-docx），本地绿（本机已装）。
- **5 Whys（带证据）**：为什么云端红本地绿？→ ci.yml 从未安装 python-docx/Pillow，本地环境恰好有。为什么 ci.yml 没装？→ 早期 SKILL.md 声称“零外部依赖”，默认了纯标准库假设。为什么假设错了？→ 引入 docx_util 时未同步更新 CI 依赖矩阵（代码先行、CI 配置滞后）。
- **当时解法**：ci.yml 加 pip install python-docx Pillow；修正“零外部依赖”表述为“python-docx/Pillow 运行依赖、PyMuPDF/rapidocr 可选函数级容错”。
- **下次如何避免**：引入任何第三方 import 的提交必须同步更新 requirements.txt + CI；CI 从第一天就该装 requirements.txt（pip 缓存后成本几乎为零）。
- **系统性模式**：依赖声明与 CI 配置漂移是“本地绿云端红”的经典根因。

5.7 其它已记录踩坑（简列，详见 development-log）

- **explorer 句柄占用**：业绩库→素材库目录更名被 explorer 打开占用，COM 关闭窗口后成功（10-02）。
- **Word COM 原生崩溃**：gen_py 类型库缓存过期（Python 3.14 下）→ 删 %TEMP%\gen_py\<ver>\ 重建；Quit 不退出进程 → PID taskkill + 等待句柄。
- **PowerShell 重命名正则**：-replace '\.png\$', '_P1.png' 把 _2.png 改坏成 _2_P1.png → 匹配 _2\.png\$ 或全名。
- **docx 遍历 id() 去重误杀**：Python 对象 id 复用致单元格/段落被跳过 → 改用 XML XPath 路径（getpath）去重。
- **lxml iter() 遍历中删除**：删除未访问节点跳过后续节点 → 先收集 targets 再统一删除。
- **python-docx doc.paragraphs.index(p) 失效**：属性每次访问生成新对象 → 一次性取列表 / w:p 元素计数实时定位。
- **LibreOffice 转换悬空图片关系**：rels 指向缺失 media → strip v3 以“最终 zip 中 Target 是否存在”为唯一判据重建 rels。

- **docxtpl 行循环语义与直觉相反**: { %tr for %} 必须独占行、endfor 行必须有 w:t
——评估结论不迁移（模板零改造/原文契约冲突，见 10-04 ⑨）。

6. 改进动作 (Deploy)

动作	优先级	验收标准	验证方式	状态
M7 四道检查 (格式/占位/敏感/word_op en+页数+frozen) 全链路常态化	P0	任何 docx 交付前必须过 m7-check 四道 ok	真实商务标 10 文件实测 ok=true; Word 可打开	✅ 已落地 (10-04/10-05)
双通道差异逐项明示裁决 (v2.4)	P0	对话上下文逐项列示差异 + 每项附建议; 裁决入已裁决事项块	示例市 2026 实跑: 5 项差异逐项裁决记录	✅ 已落地 (10-07)
裁决检查点程序化: diffs 非空且无 verdicts → 拒绝落盘	P1	tender-parse 强制校验	新增校验用例: 无 verdicts 拒绝	🕒 待开发
写后回读断言标准化 (save/register 后 load 对比)	P1	所有权威源写入路径带回读单测	test_feedback_register 已回读; 新写入路径同规则	◆ 部分 (register/trash/query 已补, 其余待推广)
真实数据	P1	数据缺失 → 整模块	8 个真实用例已修;	◆ 部分

动作	优先级	验收标准	验证方式	状态
用例前置 依赖清单 化 (require_data)		/单用例 SkipTest 而非报错	新用例沿用	
OCR 低置信字段强制二次核对 (日期/姓名/编号)	P1	归档与简历填充流程含“核对检查点”	荣誉日期铁律已固化; 简历 OCR 三重清洗	◆ 部分 (仍有手工依赖)
Word 环境检查工具化 (bidcraft env-check)	P1	一键检测依赖/COM/gen_py 缓存/句柄占用	新模块 m_env_check + 冒烟测试	⌚ 待开发
需求对齐模板加“本质定位”节	P2	每个新模块需求对齐含“是什么/不是什么/谁消费”	复核 M2/M4 文档结构; 新模块沿用	⌚ 待开发
模板需求规格口述→文档化前置	P2	模板制作前有用户确认的规格文档	M2 流程检查点	⌚ 待开发
数据一致性巡检 (台账/磁盘/回收站三方对)	P2	新 CLI 巡检命令 + 单测	inspect 扩展或新命令	⌚ 待开发

动作	优先级	验收标准	验证方式	状态
账)				
gen_py 缓 存清理工 具化	P2	env-check 内一键清 理并重建	Word COM 冒烟通 过	🕒 待开发

7. 自评表（评分制 v1.1，专业版）

#	硬标准	得分(0-2)	说明
1	每个判断有数据/事例 支撑	2	全文事件+日期+ 数据（339 测 试/33 里程碑/90 页 PDF 等）
2	含开发过程总结（方法 论/演进/决策脉络）	2	§3.1 方法论 + §3.2 架构决策演 进
3	指出流程问题 + 可复现 改进建议	2	§3.3 六项流程摩 擦 + §3.4 六条可 复现建议
4	双版本均产出 PDF	2	专业版 + 大白话 版，同取证同结 构
5	多带实例（事件+日期+ 数据）	2	每条均回指 development-log 日期条目
6	专业名词详细解释	2	项目特有术语均 加一句说明 （D25/sectPr/双 通道等）
7	≥3 做对了 + ≥3 踩坑，	2	5 做对了 + 7 踩

#	硬标准	得分(0-2)	说明
	带证据		坑
8	识别 ≥2 系统性模式	2	Word 引用完整性/Windows 编码/静默不落盘/OCR 低置信
9	三层提炼 (成绩→做法→经验)	2	§4 每条含“原来难在哪/改了哪环节/换环境照做”
10	5 Whys 每步带证据 + 多因分支 + 未证实标注	2	§5 主坑均 5 问带证据; 5.2/5.4 多因分支; 无未证实项
11	改进动作有优先级/验收/验证方式	2	§6 表含三列全部字段
12	无指责个人语言	2	全部系统归因 (“系统允许了 X”/ “流程未设检查点”)
13	量化数据真实取自日志/测试/CI	2	339 项为当日实跑; 耗时取自 development-log
14	篇幅聚焦坑+根因+改进	2	无过程流水账; 时间线简表化
15	含关键事件时间线简表	2	§2 表格
总分		30/30	待用户复核

草稿版本: v1.0 (2026-10-07) 。取证源: development-log.md 1217 行全文 + git log 130 条 + 三塔层实跑 339 项。